

Retrieval-Augmented QA System Evaluation Report

March 2026

1 Introduction

This report summarizes the evaluation of our Retrieval-Augmented Generation (RAG) system for document-based question answering. The goal of this evaluation was to measure retrieval quality, latency performance, and the effect of system improvements such as caching and LLM-based re-ranking.

The system was tested using a benchmark dataset of **110 queries**. Each query is associated with the document chunks that contain the correct information, allowing us to compute retrieval metrics such as Recall@K, Top-1 Accuracy, MRR, and nDCG.

The evaluation also includes stage-wise latency measurements across the pipeline.

2 System Architecture

The system follows a standard Retrieval-Augmented Generation pipeline:

User Query → Query Rewrite → Query Embedding → Vector Retrieval → Re-ranking →
Context Construction → Answer Generation

Components

- **Embedding Model:** Converts queries into dense vector representations.
- **Vector Store (FAISS):** Retrieves semantically similar document chunks.
- **Re-ranking Model:** Uses an LLM to improve ranking quality.
- **Context Builder:** Selects top chunks and constructs the prompt context.
- **Answer Generator:** Produces the final answer using the LLM.

3 Retrieval Evaluation Metrics

The retrieval system was evaluated on 110 benchmark queries.

3.1 Baseline Retrieval (No Optimization)

Metric	Value
Recall@3	73.6%
Top-1 Accuracy	52.7%
Top-3 Accuracy	73.6%
MRR	0.6767
nDCG	0.7512
False Positive Rate	0%

3.2 Retrieval with Re-ranking

Metric	Value
Recall@3	85.8%
Top-1 Accuracy	72.3%
Top-3 Accuracy	85.8%
MRR	0.8136
nDCG	0.8461
False Positive Rate	0%

3.3 Impact of Re-ranking

Re-ranking improves retrieval performance significantly:

- Recall@3 improves by approximately **12%**.
- Top-1 accuracy improves by approximately **19%**.
- Ranking quality (MRR and nDCG) improves substantially.

This indicates that the initial retrieval step finds relevant chunks, but re-ranking helps surface the most relevant chunk at the top.

4 Caching Strategy

Two types of caching were considered during evaluation.

4.1 Embedding Cache

Query embeddings can be cached for repeated queries.

- Query $\rightarrow$ embedding vector
- Eliminates repeated embedding computation
- Reduces compute cost

However, embedding time is already very small (around 14ms), so the latency improvement is minimal.

4.2 Retrieval Cache

Top-K retrieval results can also be cached.

- Query $\rightarrow$ retrieved chunk IDs
- Eliminates vector search for repeated queries
- Improves performance in production workloads

In our benchmark dataset all queries were unique, so caching did not affect evaluation metrics.

5 Latency Benchmark

Latency was measured for each stage of the pipeline.

Stage	Mean Latency	P95 Latency
Embedding	0.014 s	0.009 s
Retrieval	0.266 s	0.324 s
Re-ranking	1.033 s	1.985 s
Generation	2.045 s	4.086 s

5.1 Observations

- Embedding time is negligible compared to other stages.
- Retrieval using FAISS is relatively fast.
- Re-ranking adds noticeable latency but improves ranking quality.
- Generation time is the largest contributor to total latency.

Estimated end-to-end latency:

- Without re-ranking: ~ 2.3 seconds
- With re-ranking: ~ 3.3 seconds

6 Trade-off: Accuracy vs Latency

Re-ranking improves retrieval accuracy but increases latency.

Metric	Baseline	With Re-ranking
Recall@3	73.6%	85.8%
Top-1 Accuracy	52.7%	72.3%
MRR	0.676	0.814
Latency	2.3s	3.3s

This shows a clear trade-off between system speed and answer quality.

7 Observations

Some key observations from the evaluation:

- The retrieval pipeline is effective but still below the target Recall@3 of 90%.
- Re-ranking significantly improves ranking quality.
- Generation latency dominates overall system runtime.
- FAISS-based retrieval performs efficiently even with multiple queries.
- The system produced no false positives in the benchmark dataset.

8 Planned Improvements

To further improve the system, the following steps are planned:

- Implement hybrid retrieval combining vector search and keyword search.
- Increase candidate retrieval before re-ranking.
- Explore stronger embedding models.
- Introduce smarter re-ranking strategies to reduce latency.
- Expand evaluation to include hallucination detection and entity coverage metrics.

9 Conclusion

The current system demonstrates strong retrieval performance and reasonable latency. Re-ranking significantly improves answer quality but introduces additional latency.

Future improvements will focus on improving recall while reducing response time so that the system becomes both accurate and production-ready.